COMPUTER VISION

Subject Code: 21CSE454T

ASSIGNMENT - UNIT 2

Edge & Point Detection

Task:

Multi-stage boundary extraction. Implement Canny Edge Detection and a Harris Corner Detector on a complex structural image (like a checkerboard or architectural building) to analyze local gradient intensities and highlight key interest points.

SUBMITTED BY

Name	**:**	Srevarshan
Register Number	**:**	RA2311026050039
Year	**:**	4th Year
Department	**:**	CSE - AIML
Section	**:**	A
Subject	**:**	Computer Vision
Subject Code	**:**	21CSE454T

1. AIM

To detect edges using the Canny edge detector and corner points using the Harris corner detector on a building image and a checkerboard image.

2. ALGORITHM / PROCEDURE

A. Canny Edge Detection

Step 1: Convert the image to grayscale.

Step 2: Smooth the image with a 5 x 5 Gaussian filter to reduce noise.

Step 3: Find the gradients Gx and Gy using the Sobel operator, then the gradient magnitude and direction.

Step 4: Non-maximum suppression: keep only the pixels that are a local maximum along the gradient direction.

Step 5: Double threshold: mark pixels as strong edges, weak edges or non-edges.

Step 6: Hysteresis: keep a weak edge only if it is connected to a strong edge.

B. Harris Corner Detection

Step 1: Find the image derivatives Ix and Iy.

Step 2: Compute Ix^2, Iy^2 and IxIy and smooth them with a Gaussian window.

Step 3: Compute the corner response $R = \det(M) - k (\text{trace } M)^2$ with $k = 0.04$.

Step 4: Keep the points where R is above a threshold and is a local maximum.

Step 5: Mark the corners on the image.

3. PYTHON CODE

File name: unit2_edge_corner.py

```
"""
Unit 2 - Edge & Point Detection
Multi-stage Canny edge detection and Harris corner detection on structural
images (architectural facade + perspective-warped checkerboard).
"""
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "outputs")
os.makedirs(OUT, exist_ok=True)


# ---------------- Input images ----------------
def make_checkerboard(n=8, sq=50):
    """Synthetic checkerboard, warped by a perspective transform (adds complexity)."""
    board = np.kron((np.indices((n, n)).sum(0) % 2), np.ones((sq, sq))) * 255
    board = np.pad(board.astype(np.uint8), 60, constant_values=127)
    h, w = board.shape
    src = np.float32([[0, 0], [w, 0], [w, h], [0, h]])
    dst = np.float32([[60, 30], [w - 20, 80], [w - 70, h - 10], [10, h - 60]])
    M = cv2.getPerspectiveTransform(src, dst)
    return cv2.warpPerspective(board, M, (w, h), borderValue=127)


building = cv2.imread(os.path.join(HERE, "building.jpg"))
checker = cv2.cvtColor(make_checkerboard(), cv2.COLOR_GRAY2BGR)
cv2.imwrite(os.path.join(OUT, "input_checkerboard.png"), checker)


# ---------------- Canny: explicit stages ----------------
def canny_stages(gray, low=50, high=150, sigma=1.4):
    # Stage 1: noise reduction
    blur = cv2.GaussianBlur(gray, (5, 5), sigma).astype(np.float64)
    # Stage 2: intensity gradients (Sobel)
    gx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)
    mag = np.hypot(gx, gy)
    ang = (np.rad2deg(np.arctan2(gy, gx)) + 180) % 180
    # Stage 3: non-maximum suppression (thin edges to 1 px)
    H, W = mag.shape
    nms = np.zeros_like(mag)
    q = np.zeros((4, H, W))
    m = np.pad(mag, 1)
    q[0] = np.maximum(m[1:-1, 2:], m[1:-1, :-2])       # 0 deg   (left / right)
    q[1] = np.maximum(m[:-2, 2:], m[2:, :-2])          # 45 deg
    q[2] = np.maximum(m[:-2, 1:-1], m[2:, 1:-1])       # 90 deg  (up / down)
    q[3] = np.maximum(m[:-2, :-2], m[2:, 2:])          # 135 deg
    bin_ = (((ang + 22.5) // 45) % 4).astype(int)
    neigh = np.take_along_axis(q, bin_[None], 0)[0]
    nms[mag >= neigh] = mag[mag >= neigh]
    # Stage 4: double threshold
    strong = nms >= high
    weak = (nms >= low) & ~strong
    # Stage 5: hysteresis - keep weak edges connected to strong ones
    n, labels = cv2.connectedComponents((strong | weak).astype(np.uint8), connectivity=8)
    keep = np.zeros(n, bool)
    keep[np.unique(labels[strong])] = True
    keep[0] = False
    edges = keep[labels].astype(np.uint8) * 255
    return dict(blur=blur, gx=gx, gy=gy, mag=mag, nms=nms,
                thresh=(strong * 255 + weak * 100).astype(np.uint8), edges=edges)
```

```
# ---------------- Harris corner detector ----------------
def harris(gray, k=0.04, block_sigma=1.5, thresh_ratio=0.01, pre_sigma=0, nms_size=7):
    g = gray.astype(np.float64)
    if pre_sigma:                                       # suppress fine texture (e.g. bricks)
        g = cv2.GaussianBlur(g, (0, 0), pre_sigma)
    Ix = cv2.Sobel(g, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(g, cv2.CV_64F, 0, 1, ksize=3)
    # Structure tensor M = sum w(x,y) [Ix^2 IxIy; IxIy Iy^2]
    Sxx = cv2.GaussianBlur(Ix * Ix, (0, 0), block_sigma)
    Syy = cv2.GaussianBlur(Iy * Iy, (0, 0), block_sigma)
    Sxy = cv2.GaussianBlur(Ix * Iy, (0, 0), block_sigma)
    R = (Sxx * Syy - Sxy**2) - k * (Sxx + Syy) ** 2       # det(M) - k*trace(M)^2
    # Threshold + non-maximum suppression -> clean key points
    local_max = R == cv2.dilate(R, np.ones((nms_size, nms_size)))
    pts = np.argwhere(local_max & (R > thresh_ratio * R.max()))
    return R, pts


def run(name, bgr, low, high, **harris_args):
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    st = canny_stages(gray, low, high)
    cv_edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 1.4), low, high, L2gradient=True)
    agree = (st["edges"] == cv_edges).mean() * 100

    R, pts = harris(gray, **harris_args)
    cvR = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)

    vis = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR) // 2 + bgr // 2
    for y, x in pts:
        cv2.circle(vis, (int(x), int(y)), 4, (0, 0, 255), 2)
    over = bgr.copy()
    over[st["edges"] > 0] = (0, 255, 0)

    print(f"[{name}] size={gray.shape[1]}x{gray.shape[0]}")
    print(f"  Gradient magnitude : mean={st['mag'].mean():.1f}  max={st['mag'].max():.1f}")
    print(f"  Canny thresholds   : low={low}, high={high}")
    print(f"  Edge pixels        : after NMS={int((st['nms'] > low).sum())},
        final={int((st['edges'] > 0).sum())}"
          f" ({(st['edges'] > 0).mean() * 100:.2f}% of image)")
    print(f"  Agreement with cv2.Canny : {agree:.2f}% of pixels")
    print(f"  Harris parameters        : {harris_args}")
    print(f"  Harris corners detected  : {len(pts)}  (R max={R.max():.3g}, cv2 R
        max={cvR.max():.3g})")

    # Figure A: Canny stages
    if name == "Building":
        fig, ax = plt.subplots(4, 2, figsize=(12, 15))
    else:
        fig, ax = plt.subplots(2, 4, figsize=(17, 9.5))
    panels = [("1. Grayscale input", gray, "gray"), ("2. Gaussian smoothed", st["blur"],
        "gray"),
              ("3a. Gradient Gx (Sobel)", st["gx"], "RdBu"), ("3b. Gradient Gy (Sobel)",
                  st["gy"], "RdBu"),
              ("3c. Gradient magnitude", st["mag"], "inferno"), ("4. Non-max suppression",
                  st["nms"] > 0, "gray"),
              ("5. Double threshold\n(white=strong, grey=weak)", st["thresh"], "gray"),
              ("6. Hysteresis - final edges", st["edges"], "gray")]
    for a, (t, im, cm) in zip(ax.ravel(), panels):
        a.imshow(im, cmap=cm); a.set_title(t, fontsize=11); a.axis("off")
    fig.suptitle(f"Canny multi-stage pipeline - {name}", fontsize=14, fontweight="bold")
    plt.tight_layout(); plt.savefig(os.path.join(OUT, f"{name.lower()}_canny_stages.png"),
        dpi=110); plt.close()

    # Figure B: final results
    if name == "Building":
        fig, ax = plt.subplots(3, 1, figsize=(10, 17))
    else:
        fig, ax = plt.subplots(1, 3, figsize=(18, 6.3))
    ax[0].imshow(cv2.cvtColor(over, cv2.COLOR_BGR2RGB)); ax[0].set_title("Canny edges overlaid
```

```
         (green)")
    r = np.cbrt(R)
    lim = np.abs(r).max()
    ax[1].imshow(r, cmap="RdBu_r", vmin=-lim, vmax=lim)
    ax[1].set_title("Harris response R (red = corner, blue = edge, white = flat)")
    ax[2].imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)); ax[2].set_title(f"Harris corners:
        {len(pts)} key points (red)")
    for a in ax: a.axis("off")
    plt.tight_layout(); plt.savefig(os.path.join(OUT, f"{name.lower()}_results.png"),
        dpi=110); plt.close()
    cv2.imwrite(os.path.join(OUT, f"{name.lower()}_corners.png"), vis)
    cv2.imwrite(os.path.join(OUT, f"{name.lower()}_edges.png"), st["edges"])


run("Building", building, 60, 160, thresh_ratio=0.02, pre_sigma=2.5, block_sigma=2.5,
    nms_size=11)
print()
run("Checkerboard", checker, 50, 150, thresh_ratio=0.01)
print("\nAll figures saved in", OUT)
```

4. INPUT IMAGE

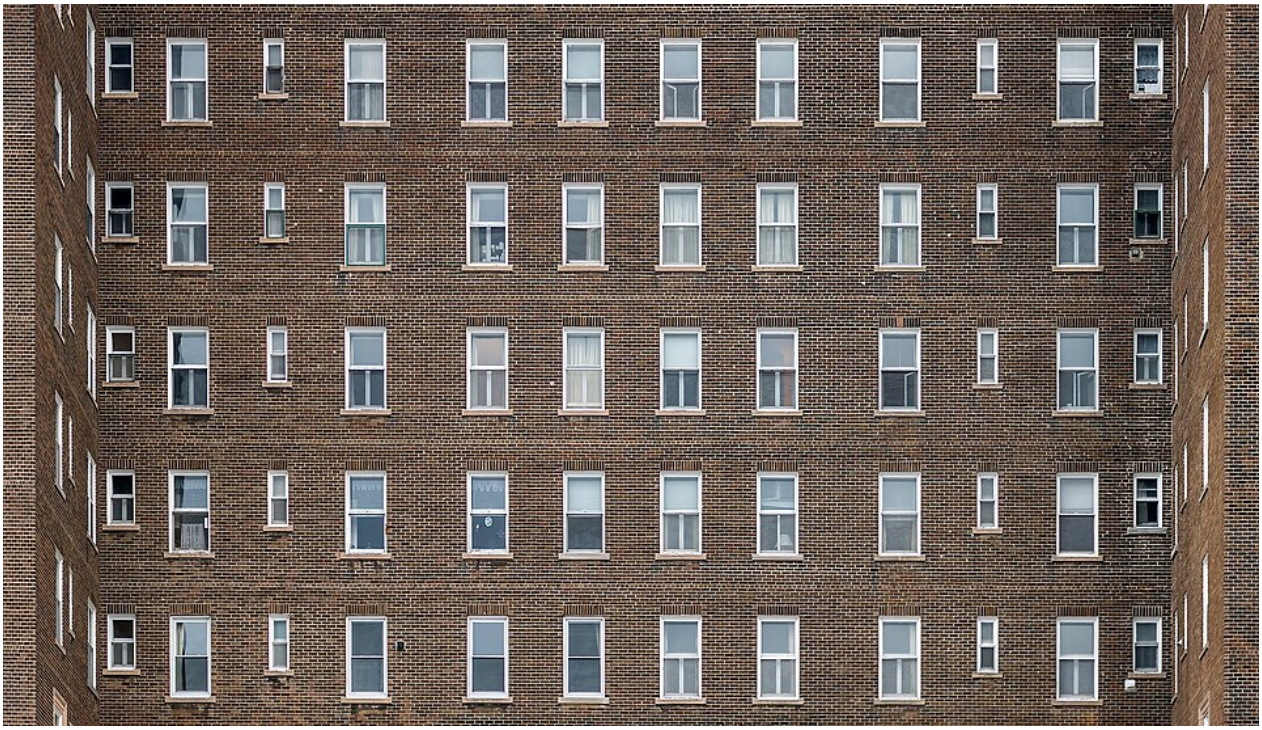


Figure 1: Input image 1 - building with a grid of windows. Source: Wikimedia Commons, by Wilfredor, CC BY-SA 4.0

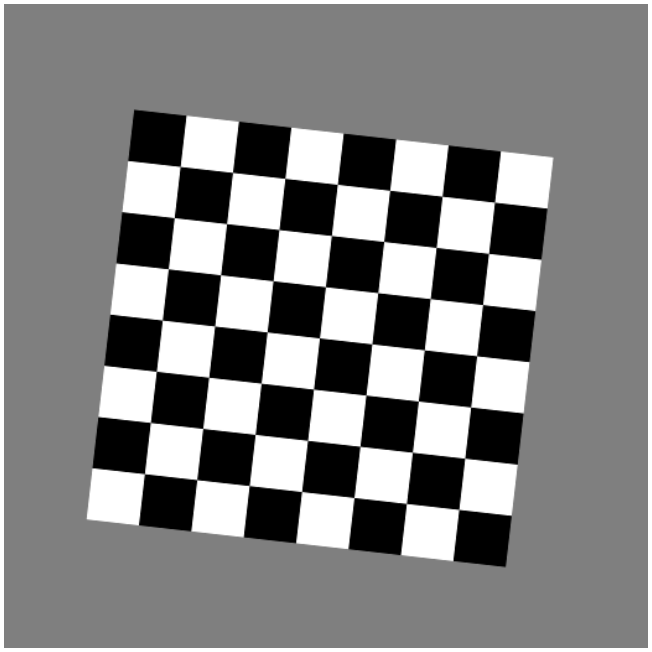


Figure 2: Input image 2 - checkerboard with perspective distortion (generated in the program)

5. OUTPUT SCREENSHOTS

```
Terminal
$ python unit2_edge_corner.py
[Building] size=960x551
  Gradient magnitude : mean=60.7  max=512.8
  Canny thresholds   : low=60, high=160
  Edge pixels        : after NMS=57052, final=26224 (4.96% of image)
  Agreement with cv2.Canny : 98.89% of pixels
  Harris parameters        : {'thresh_ratio': 0.02, 'pre_sigma': 2.5, 'block_sigma': 2.5, 'nms_size': 11}
  Harris corners detected  : 434  (R max=6.48e+07, cv2 R max=4.69e+08)

[Checkerboard] size=520x520
  Gradient magnitude : mean=38.1  max=560.9
  Canny thresholds   : low=50, high=150
  Edge pixels        : after NMS=5721, final=5721 (2.12% of image)
  Agreement with cv2.Canny : 99.83% of pixels
  Harris parameters        : {'thresh_ratio': 0.01}
  Harris corners detected  : 81  (R max=6.5e+10, cv2 R max=3.06e+08)

All figures saved in Unit2_Edge_Point_Detection/outputs
```

Figure 3: Program output in the terminal

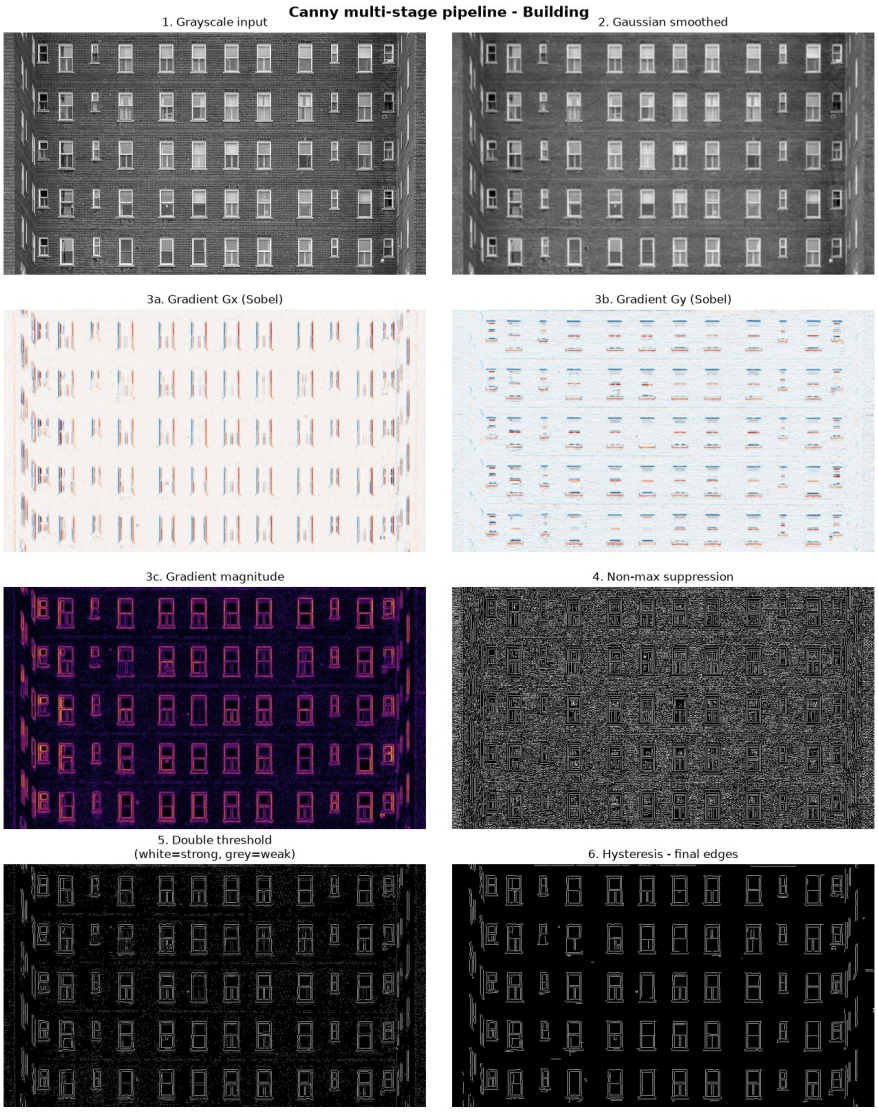


Figure 4: Stages of Canny edge detection - building

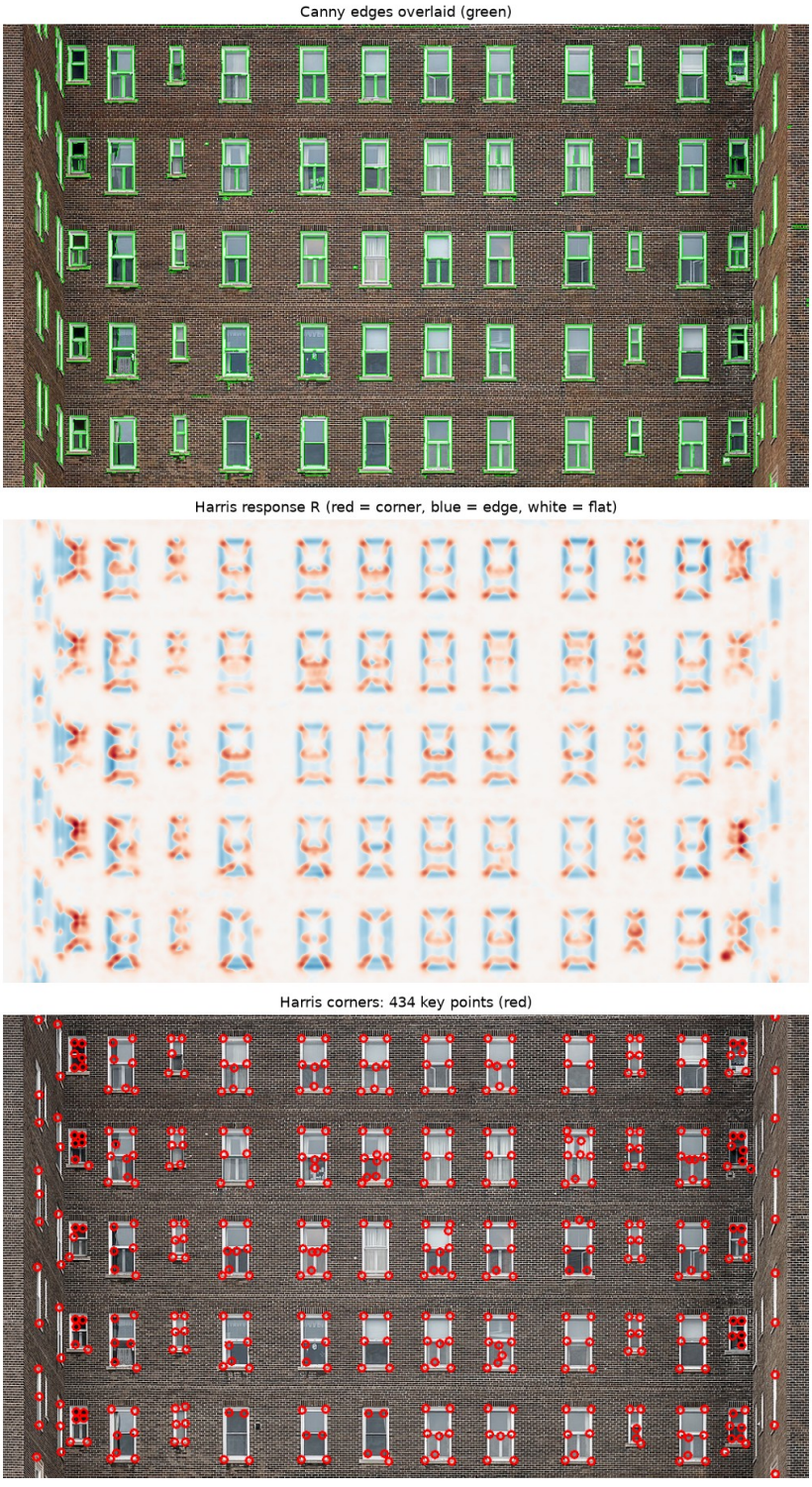


Figure 5: Building - Canny edges, Harris response and corners

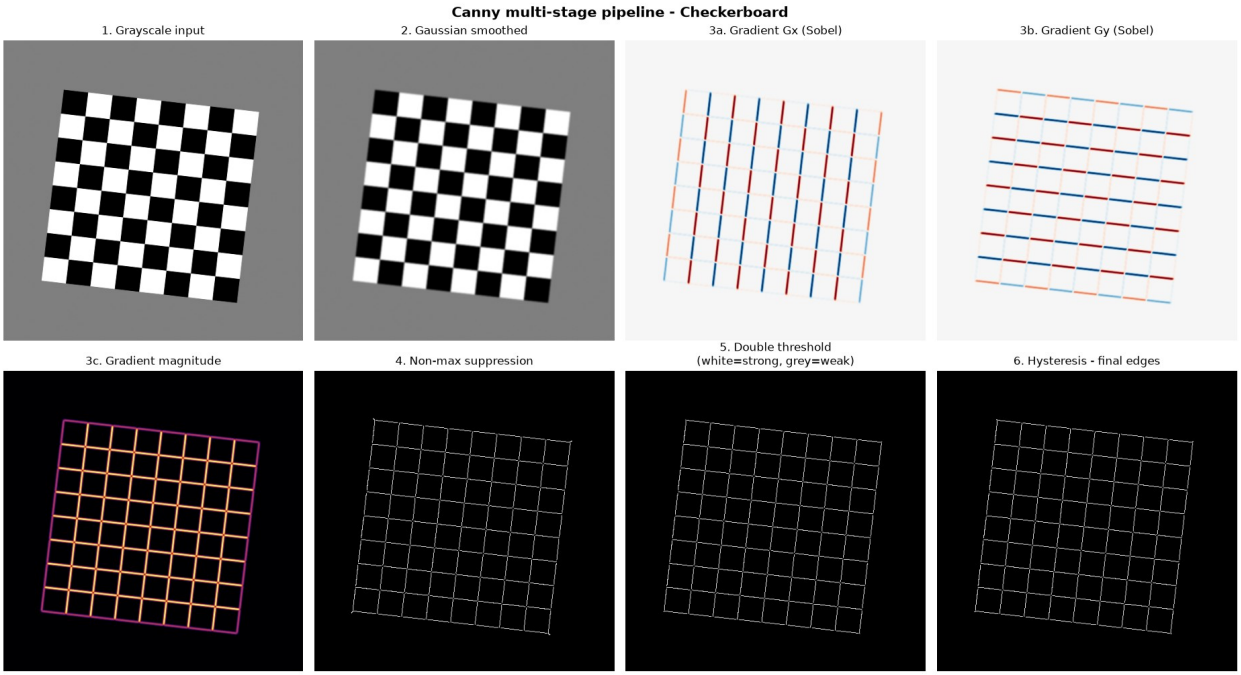


Figure 6: Stages of Canny edge detection - checkerboard

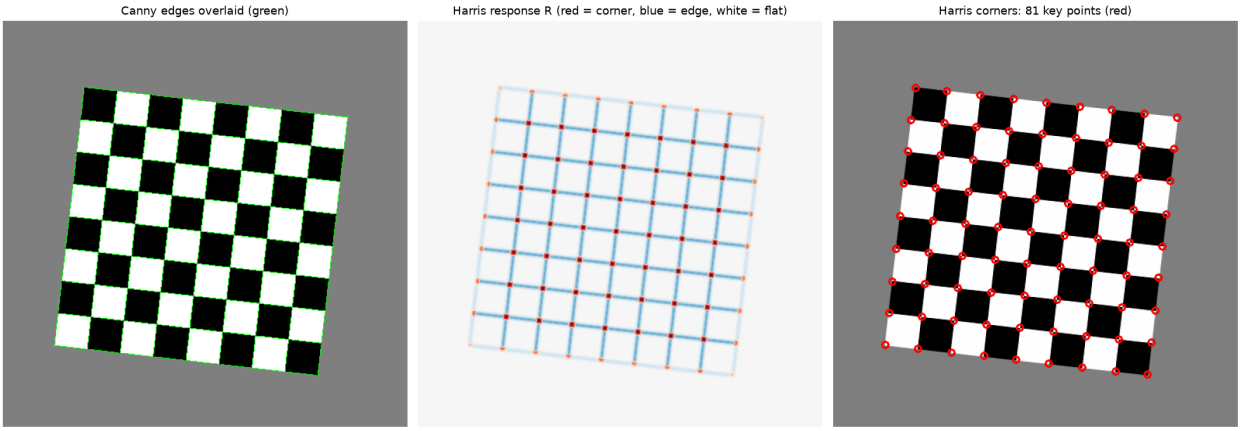


Figure 7: Checkerboard - Canny edges, Harris response and corners

6. RESULT ANALYSIS / OBSERVATIONS

Result	Building	Checkerboard
Mean gradient magnitude	60.7	38.1
Edge pixels after NMS	57,052	5,721
Edge pixels after hysteresis	26,224	5,721
Match with OpenCV Canny	98.89 %	99.83 %
Harris corners found	434	81

- The building has a higher mean gradient because the brick wall has a lot of texture.
- Hysteresis removed more than half of the edge pixels in the building image. These were mostly weak edges from the bricks. The window frames were kept.
- All 81 corners of the checkerboard (9 x 9 grid) were found, even with the perspective distortion.
- In the building image, the corners were found mostly at the window corners after light smoothing.
- The Harris response is high at corners, negative along edges and close to zero in flat areas.

7. CONCLUSION

Canny edge detection gave thin and continuous edges, and its multi-stage process removed most of the texture noise. The Harris detector found the important corner points in both images. Edges have a strong gradient in one direction, while corners have strong gradients in two directions.